Memorandum

To: Charlie Refvem, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Lucas Nathan
lrnathan@calpoly.edu

Vandan Patel
vpatel43@calpoly.edu

Oliver Lenoir
olenoir@calpoly.edu

Date: 10 February 2026

RE: **Lab 0x03 - Closed Loop Control and the Scheduler**

Overview:

The objective of this lab was to implement a closed-loop motor control system using a cooperative task scheduler. Hardware drivers for the motor and encoder were integrated with multiple scheduled tasks to perform motor actuation, sensor feedback, and user interaction. A step input was initiated through a user interface task, and encoder position data was collected over time to evaluate the closed-loop response. The system was designed to demonstrate stable behavior rather than optimal tuning. Collected data was streamed over the serial interface, logged to a CSV file, and post-processed using a Python script adapted from a previous assignment to generate a step response plot for analysis.

Step Response:

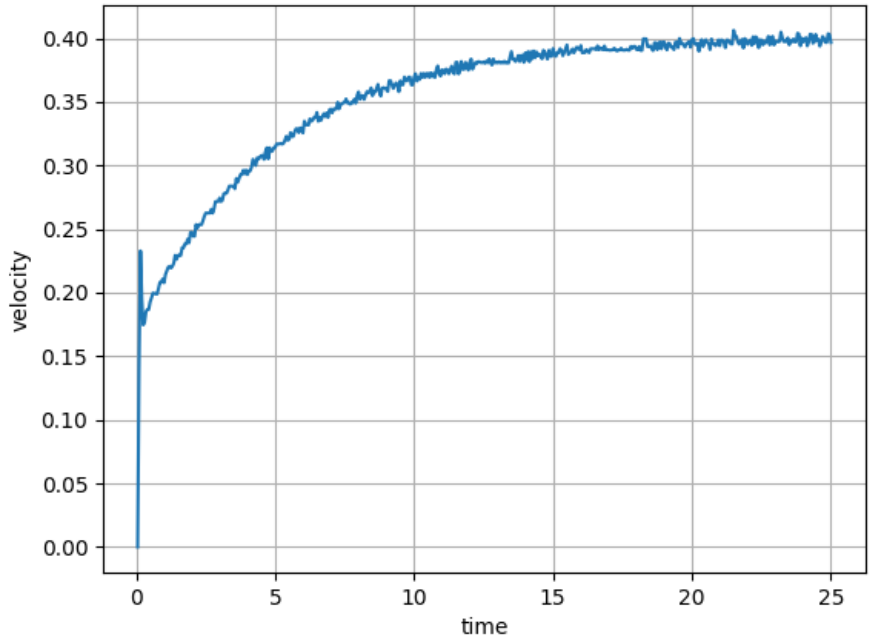


Figure 1. Closed-loop step response of the motor velocity following a commanded step input. The response rises smoothly from rest and asymptotically approaches a steady-state velocity, demonstrating stable closed-loop behavior with minimal oscillation

Discussion

Figure 1 shows the measured closed-loop response of the motor velocity following a commanded step input. After the start command is issued through the user interface task, the motor accelerates smoothly from rest and converges to a steady-state velocity without sustained oscillation or instability. Although the controller gains were not tuned for optimal transient performance, the response demonstrates stable closed-loop behavior and proper coordination between the scheduler, motor task, and encoder feedback. These results confirm that the cooperative task scheduler successfully supports closed-loop control while simultaneously managing user input and data collection.

While the response is slow, this is due to our choice of KI and KP, which we could have made significantly higher to see a faster response, but we thought it was easiest to visualize on a longer time scale. Once implemented into the rest of our code, we will increase the constants to get a faster response time from our ROMI.

Attachment A:

Source code:

Main.py:

```
from motor_driver import motor_driver
from encoder      import encoder
from task_motor   import task_motor
from task_user    import task_user
from task_share   import Share, Queue, show_all
from cotask       import Task, task_list
from gc           import collect
from pyb import Timer,Pin

# Build all driver objects first



# PWM timers
tim3 = Timer(3, freq=20000)
tim4 = Timer(4, freq=20000)

pwm_L = tim3.channel(1, Timer.PWM, pin=Pin.cpu.C6)
pwm_R = tim4.channel(2, Timer.PWM, pin=Pin.cpu.B7)

# Motors
leftMotor  = motor_driver(pwm_L, Pin.cpu.B9,  Pin.cpu.B8)
rightMotor = motor_driver(pwm_R, Pin.cpu.H0, Pin.cpu.H1)

# Encoders
leftEncoder  = encoder(1, Pin.cpu.A8, Pin.cpu.A9)
rightEncoder = encoder(2, Pin.cpu.A0, Pin.cpu.A1)
```

```
# Build shares and queues
leftMotorGo   = Share("B",     name="Left Mot. Go Flag")
rightMotorGo  = Share("B",     name="Right Mot. Go Flag")
dataValues    = Queue("f", 500, name="Data Collection Buffer")
timeValues    = Queue("L", 500, name="Time Buffer")

# Build task class objects
leftMotorTask  = task_motor(leftMotor,  leftEncoder,
                            leftMotorGo, dataValues, timeValues)
rightMotorTask = task_motor(rightMotor, rightEncoder,
                            rightMotorGo,             dataValues,
timeValues)
userTask   =   task_user(leftMotorGo,   rightMotorGo,   dataValues,
timeValues)

# Add tasks to task list
task_list.append(Task(leftMotorTask.run, name="Left Mot. Task",
                      priority = 1, period = 50, profile=True))
task_list.append(Task(rightMotorTask.run,     name="Right    Mot.
Task",
                      priority = 1, period = 50, profile=True))
task_list.append(Task(userTask.run, name="User Int. Task",
                      priority = 0, period = 0, profile=False))

# Run the garbage collector preemptively
collect()

leftMotor.enable()

# Run the scheduler until the user quits the program with Ctrl-C
```

```
while True:
    try:
        task_list.pri_sched()

    except KeyboardInterrupt:
        print("Program Terminating")
        leftMotor.disable()
        rightMotor.disable()
        break

print("\n")
print(task_list)
print(show_all())
```

Encoder.py

```
-*- coding: utf-8 -*-
"""
Created on Tue Jan 27 09:38:37 2026


@author: orlen
"""


from time import ticks_us, ticks_diff   # Use to get dt value in
update()
from array import array
from pyb import Pin, Timer
class encoder:
    '''A quadrature encoder decoding interface encapsulated in a
Python class'''


    def __init__(self, tim, chA_pin, chB_pin):
        '''Initializes an Encoder object'''
        self.tim= Timer(tim, period=0xFFFF, prescaler = 0)
        self.tim.channel(1, pin=chA_pin, mode=Timer.ENC_AB)
        self.tim.channel(2, pin=chB_pin, mode=Timer.ENC_AB)


        self.position    = 0      # Total accumulated position of
the encoder
        self.prev_position = 0   # Counter value from the most
recent update
        self.delta       = 0      # Change in count between last
two updates
        self.dt           = 0      # Amount of time between last
two updates
        self.earlier    = 0
        self.now        = 0
```

```
    def update(self):
        self.position = -self.tim.counter()
        self.delta = self.position - self.prev_position
        self.prev_position = self.position

        if self.delta > 32768:
            self.delta -= 65536
        elif self.delta < -32768:
            self.delta += 65536

        self.now = ticks_us()
        self.dt = ticks_diff(self.now, self.earlier)
        self.earlier = self.now

    def get_position(self):

        return self.position

    def get_velocity(self):

        return self.delta/self.dt

    def zero(self):
        self.prev_position = self.tim.counter()
        self.position = 0
pass
```

task_user.py:

```
''' This file demonstrates an example UI task using a custom class with a
    run method implemented as a generator
'''
from pyb import USB_VCP
from task_share import Share
import micropython


S0_INIT = micropython.const(0) # State 0 - initialiation
S1_CMD  = micropython.const(1) # State 1 - wait for character input
S2_COL  = micropython.const(2) # State 2 - wait for data collection to end
S3_DIS  = micropython.const(3) # State 3 - display the collected data


UI_prompt = ">: "


class task_user:
    '''
    A class that represents a UI task. The task is responsible for reading user
    input over a serial port, parsing the input for single-character commands,
    and then manipulating shared variables to communicate with other tasks based
    on the user commands.
    '''


    def __init__(self, leftMotorGo, rightMotorGo, dataValues, timeValues):
        '''
```

```
        Initializes a UI task object


        Args:
            leftMotorGo (Share):  A share object representing a boolean flag to
                                  start data collection on the left motor
            rightMotorGo (Share): A share object representing a boolean flag to
                                  start data collection on the right motor
            dataValues (Queue):   A queue object used to store collected encoder
                                  position values
            timeValues (Queue):   A queue object used to store the time stamps
                                  associated with the collected encoder data
        '''


        self._state: int            = S0_INIT       # The present state


        self._leftMotorGo: Share  = leftMotorGo  # The "go" flag to start data
                                                 #     collection from the left
                                                 #   motor   and encoder pair


        self._rightMotorGo: Share = rightMotorGo # The "go" flag to start data
                                                 #     collection from the right
```

```
                                                    #   motor   and
encoder pair

        self._ser: stream         = USB_VCP()     # A serial port
object used to
                                                    #           read
character entry and to
                                                    # print output

        self._dataValues: Queue    = dataValues    # A reusable
buffer for data
                                                    # collection

        self._timeValues: Queue    = timeValues    # A reusable
buffer for time
                                                    #       stamping
collected data

        self._ser.write("User Task object instantiated")

    def run(self):
        '''
        Runs one iteration of the task
        '''

        while True:

            if self._state == S0_INIT: # Init state (can be
removed if unneeded)
                self._ser.write("Initializing user task\r\n")
                self._ser.write("Waiting for go command: 'l' for
left, 'r' for right\r\n")
                self._ser.write(UI_prompt)
```

```
                self._state = S1_CMD

            elif self._state == S1_CMD: # Wait for UI commands
                # Wait for at least one character in serial
buffer
                if self._ser.any():
                    # Read the character and decode it into a
string
                    inChar = self._ser.read(1).decode()
                    # If the character is an upper or lower case
"l", start data
                    # collection on the left motor and if it is
an "r", start
                    # data collection on the right motor
                    if inChar in {"l", "L"}:
                        self._ser.write(f"{inChar}\r\n")
                        self._leftMotorGo.put(True)
                        self._ser.write("Starting left motor
loop...\r\n")
                        self._ser.write("Starting data
collection...\r\n")
                        self._ser.write("Please wait... \r\n")
                        self._state = S2_COL
                    elif inChar in {"r", "R"}:
                        self._ser.write(f"{inChar}\r\n")
                        self._rightMotorGo.put(True)
                        self._ser.write("Starting right motor
loop...\r\n")
                        self._ser.write("Starting data
collection...\r\n")
                        self._ser.write("Please wait... \r\n")
                        self._state = S2_COL
```

```
            elif self._state == S2_COL:
                # While the data is collecting (in the motor task) block out the
                # UI and discard any character entry so that commands don't
                # queue up in the serial buffer
                if self._ser.any(): self._ser.read(1)

                # When both go flags are clear, the data collection must have
                # ended and it is time to print the collected data.
                if not self._leftMotorGo.get() and not self._rightMotorGo.get():
                    self._ser.write("Data collection complete...\r\n")
                    self._ser.write("Printing data...\r\n")
                    self._ser.write("--------------------\r\n")
                    self._ser.write("Time, Position\r\n")
                    self._state = S3_DIS

            elif self._state == S3_DIS:
                # While data remains in the buffer, print that data in a command
                # separated format. Otherwise, the data collection is finished.
                if self._dataValues.any():
self._ser.write(f"{self._timeValues.get()},{self._dataValues.get()},\r\n")
                else:
                    self._ser.write("--------------------\r\n")
                    self._ser.write("Waiting for go command: 'l' for left, 'r' for right\r\n")
```

```
        self._ser.write(UI_prompt)
        self._state = S1_CMD

yield self._state
```

task_motor.py:

```
''' This file demonstrates an example motor task using a custom class with a
    run method implemented as a generator
'''
from motor_driver import motor_driver
from encoder      import encoder
from task_share   import Share, Queue
from utime        import ticks_us, ticks_diff
import micropython
import math

#inital closed loop control values
v_in=.4
kp=0.5
ki=0.2
v_out=0


S0_INIT = micropython.const(0) # State 0 - initialiation
S1_WAIT = micropython.const(1) # State 1 - wait for go command
S2_RUN   = micropython.const(2)  #  State  2  -  run  closed  loop control

class task_motor:
    '''
    A  class  that  represents  a  motor  task.  The  task  is responsible for reading
    data from an encoder, performing closed loop control, and actuating a motor.
    Multiple objects of this class can be created to work with multiple motors
```

```
    and encoders.
    '''

    def __init__(self,
                 mot: motor_driver, enc: encoder,
                 goFlag: Share, dataValues: Queue, timeValues:
Queue):
        '''
        Initializes a motor task object

        Args:
            mot (motor_driver): A motor driver object
            enc (encoder):      An encoder object
            goFlag (Share):       A share object representing a
boolean flag to
                                start data collection
            dataValues (Queue): A queue object used to store
collected encoder
                                position values
            timeValues (Queue): A queue object used to store the
time stamps
                                associated with the collected
encoder data
        '''

        self._state: int         = S0_INIT    # The present state
of the task

        self._mot: motor_driver = mot        # A motor object

        self._enc: encoder      = enc        # An encoder object
```

```
        self._goFlag: Share      = goFlag      # A share object representing a
                                               #   flag   to   start data collection

        self._dataValues: Queue = dataValues # A queue object used to store
                                               # collected encoder position

        self._timeValues: Queue = timeValues # A queue object used to store the
                                               #    time    stamps associated with the
                                               # collected encoder data

        self._startTime: int     = 0            # The start time (in microseconds)
                                               #  for  a  batch  of collected data
        self.integral=0.0




        print("Motor Task object instantiated")

    def run(self):

        global v_in
        global effort
        effort=0
        '''
```

```
        Runs one iteration of the task
        '''

        while True:

            if self._state == S0_INIT: # Init state (can be
removed if unneeded)
                # print("Initializing motor task")
                self._state = S1_WAIT

            elif self._state == S1_WAIT: # Wait for "go command"
state
                if self._goFlag.get():
                    # print("Starting motor loop")

                    # Capture a start time in microseconds so
that each sample
                    # can be timestamped with respect to this
start time. The
                    # start time will be off by however long it
takes to
                    # transition and run the next state, so the
time values may
                    # need to be zeroed out again during data
processing.
                    self._startTime = ticks_us()
                    self._state = S2_RUN
                    self._enc.update()


            elif self._state == S2_RUN: # Closed-loop control
state
```

```
                self._enc.update()
                pos=self._enc.get_position()

                w_enc=self._enc.get_velocity()
                v_out=w_enc*2*math.pi*0.035*10**6/1440          #
convert encoder revolution to linear wheel sepeed

                #PID Loop

                t=ticks_us()

                self._dataValues.put(pos)

self._timeValues.put(ticks_diff(t,self._startTime))



                if v_in>0.55:
                    v_in=0.55
                elif v_in<-0.55:
                    v_in=-0.55

                r=v_in
                e=(r-v_out)*100/.55
                self.integral+= e*self._enc.dt/1000000
                if effort >100:
                    effort=kp*e
                elif effort <-100:
                    effort=kp*e
                else:
                    effort=kp*e+ki*self.integral
```

```
                self._mot.set_effort(effort)



print(f"{v_out},{(ticks_diff(t,self._startTime))/1000000}")

                time=[]
                velocity=[]
                time.append(ticks_diff(t,self._startTime))
                velocity.append(v_out)



                # When the queues are full, data collection is
over
                if self._dataValues.full():
                    # print("Exiting motor loop")
                    self._state = S1_WAIT
                    self._goFlag.put(False)

                print(velocity)
                print(time)

            yield self._state
```

task_share.py:

```
### @file task_share.py

#  This file contains classes which allow tasks to share data without the risk

#  of data corruption by interrupts.

#

#  @author JR Ridgely

#  @date   2017-Jan-01 JRR Approximate date of creation of file

#  @date   2021-Dec-18 JRR Docstrings changed to work without DoxyPyPy

#  @copyright This program is copyright (c) 2017-2023 by JR Ridgely and released

#             under the GNU Public License, version 3.0.

#

#  It is intended for educational use only, but its use is not limited thereto.

#   THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"

#   AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE

#   IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE

#   ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE

#   LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR

#   CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF

#  SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS

#  INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN

#  CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
```

```
#  ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
#  POSSIBILITY OF SUCH DAMAGE.

import array
import gc
import pyb
import micropython


## This is a system-wide list of all the queues and shared variables. It is
#  used to create diagnostic printouts.
share_list = []

## This dictionary allows readable printouts of queue and share data types.
type_code_strings = {'b' : "int8",   'B' : "uint8",
                     'h' : "int16",  'H' : "uint16",
                     'i' : "int(?)", 'I' : "uint(?)",
                     'l' : "int32",  'L' : "uint32",
                     'q' : "int64",  'Q' : "uint64",
                     'f' : "float",  'd' : "double"}


## Create a string holding a diagnostic printout showing the status of
#  each queue and share in the system.
#  @return A string containing information about each queue and share
def show_all ():
    gen = (str (item) for item in share_list)
```

```
    return '\n'.join (gen)



## Base class for queues and shares which exchange data between tasks.

#

#  One should never create an object from this class; it doesn't do anything

#   useful. It exists to implement things which are common between its child

#  classes @c Queue and @c Share.

class BaseShare:


    ## Create a base queue object when called by a child class initializer.

    #

    #   This method creates the things which queues and shares have in common.

    def __init__ (self, type_code, thread_protect = True, name = None):

        self._type_code = type_code

        self._thread_protect = thread_protect


        # Add this queue to the global share and queue list

        share_list.append (self)



## A queue which is used to transfer data from one task to another.

#

#   If parameter 'thread_protect' is @c True when a queue is created, transfers
```

```
#  of data will be protected from corruption in the case that one task might
#  interrupt another due to use in a pre-emptive multithreading environment or
#  due to one task being run as an interrupt service routine.
#
#  An example of the creation and use of a queue is as follows:
#
#  @code
#  import task_share
#
#  # This queue holds unsigned short (16-bit) integers
#  my_queue = task_share.Queue ('H', 100, name="My Queue")
#
#  # Somewhere in one task, put data into the queue
#  my_queue.put (some_data)
#
#  # In another task, read data from the queue
#  something = my_queue.get ()
#  @endcode
class Queue (BaseShare):

    ## A counter used to give serial numbers to queues for diagnostic use.
    ser_num = 0

    ## Initialize a queue object to carry and buffer data between tasks.
    #
    #  This method sets up a queue by allocating memory for the contents and
    #  setting up the components in an empty configuration.
```

```
    #
    #   Each queue can only carry data of one particular type which must be
    #    chosen from the following list. The data type is specified by a
    #  one-letter type code which is given as for the Python @c array.array
    #  type, which can be any of the following:
    #  |      |      |      |
    #  |:-----|:-----|:-----|
    #   | **b** (signed char) | **B** (unsigned char) | 8 bit integers |
    #  | **h** (signed short) | **H** (unsigned short) | 16 bit integers |
    #   | **i** (signed int) | **I** (unsigned int) | 32 bit integers (probably) |
    #   | **l** (signed long) | **L** (unsigned long) | 32 bit integers |
    #  | **q** (signed long long) | **Q** (unsigned long long) | 64 bit integers |
    #  | **f** (float) | **d** (double-precision float) | |
    #
    #  @param type_code The type of data items which the queue can hold
    #  @param size The maximum number of items which the queue can hold
    #    @param thread_protect @c True if mutual exclusion protection is used
    #    @param overwrite If @c True, oldest data will be overwritten with new
    #         data if the queue becomes full
    #  @param name A short name for the queue, default @c QueueN where @c N
    #         is a serial number for the queue
    def __init__ (self, type_code, size, thread_protect = False,
```

```
                  overwrite = False, name = None):
        # First call the parent class initializer
        super ().__init__ (type_code, thread_protect, name)

        self._size = size
        self._overwrite = overwrite
        self._name = str (name) if name != None \
            else 'Queue' + str (Queue.ser_num)
        Queue.ser_num += 1

        # Allocate memory in which the queue's data will be
stored
        try:
            self._buffer = array.array (type_code, range (size))
        except MemoryError:
            self._buffer = None
            raise
        except ValueError:
            self._buffer = None
            raise

        # Initialize pointers to be used for reading and writing
data
        self.clear ()

        # Since we may have allocated a bunch of memory, call
the garbage
        # collector to neaten up what memory is left for future
use
        gc.collect ()
```

```
    ## Put an item into the queue.
    #
    #   If there isn't room for the item, wait (blocking the calling process)
    #   until room becomes available, unless the @c overwrite constructor
    #   parameter was set to @c True to allow old data to be clobbered. If
    #  non-blocking behavior without overwriting is needed, one should call
    #   @c full() to ensure that the queue is not full before putting data
    #  into it:
    #  @code
    #     def some_task ():
    #         # Setup
    #         while True:
    #             if not my_queue.full ():
    #                 my_queue.put (create_something_to_put ())
    #             yield 0
    #  @endcode
    #  @param item The item to be placed into the queue
    #  @param in_ISR Set this to @c True if calling from within an ISR
    @micropython.native
    def put (self, item, in_ISR = False):
        # If we're in an ISR and the queue is full and we're not allowed to
        # overwrite data, we have to give up and exit
        if self.full ():
            if in_ISR:
```

```
                return

            # Wait (if needed) until there's room in the buffer for the data
            if not self._overwrite:
                while self.full ():
                    pass

        # Prevent data corruption by blocking interrupts during data transfer
        if self._thread_protect and not in_ISR:
            _irq_state = pyb.disable_irq ()

        # Write the data and advance the counts and pointers
        self._buffer[self._wr_idx] = item
        self._wr_idx += 1
        if self._wr_idx >= self._size:
            self._wr_idx = 0
        self._num_items += 1
        if self._num_items >= self._size:            # Can't be fuller than full
            self._num_items = self._size
        if self._num_items > self._max_full:          # Record maximum fillage
            self._max_full = self._num_items

        # Re-enable interrupts
        if self._thread_protect and not in_ISR:
            pyb.enable_irq (_irq_state)
```

```
    ## Read an item from the queue.
    #
    #   If there isn't anything in there, wait (blocking the calling process)
    #  until something becomes available. If non-blocking reads are needed,
    #   one should call @c any() to check for items before attempting to read
    #  from the queue. This is usually done in a low priority task:
    #  @code
    #     def some_task ():
    #         # Setup
    #         while True:
    #             if my_queue.any ():
    #                 something = my_queue.get ()
    #                 do_something_with (something)
    #             # More loop stuff
    #             yield 0
    #  @endcode
    #  @param in_ISR Set this to @c True if calling from within an ISR
    @micropython.native
    def get (self, in_ISR = False):
        #  Wait  until  there's  something  in  the  queue  to  be returned
        while self.empty ():
            pass

        # Prevent data corruption by blocking interrupts during data transfer
        if self._thread_protect and not in_ISR:
```

```
            irq_state = pyb.disable_irq ()

        # Get the item to be returned from the queue
        to_return = self._buffer[self._rd_idx]

        # Move the read pointer and adjust the number of items
in the queue
        self._rd_idx += 1
        if self._rd_idx >= self._size:
            self._rd_idx = 0
        self._num_items -= 1
        if self._num_items < 0:
            self._num_items = 0

        # Re-enable interrupts
        if self._thread_protect and not in_ISR:
            pyb.enable_irq (irq_state)

        return (to_return)



    ## Check if there are any items in the queue.
    #
    #  Returns @c True if there are any items in the queue and
@c False
    #  if the queue is empty.
    #  @return @c True if items are in the queue, @c False if
not
    @micropython.native
    def any (self):
        return (self._num_items > 0)
```

```
    ## Check if the queue is empty.
    #
    #  Returns @c True if there are no items in the queue and @c
False if
    #  there are any items therein.
    #  @return @c True if queue is empty, @c False if it's not
empty
    @micropython.native
    def empty (self):
        return (self._num_items <= 0)



    ## Check if the queue is full.
    #
    #  This method returns @c True if the queue is already full
and there
    #  is no room for more data without overwriting existing
data.
    #  @return @c True if the queue is full
    @micropython.native
    def full (self):
        return (self._num_items >= self._size)



    ## Check how many items are in the queue.
    #
    #  This method returns the number of items which are
currently in the
    #  queue.
    #  @return The number of items in the queue
```

```
    @micropython.native
    def num_in (self):
        return (self._num_items)



    ## Remove all contents from the queue.
    def clear (self):
        self._rd_idx = 0
        self._wr_idx = 0
        self._num_items = 0
        self._max_full = 0



    ## This method puts diagnostic information about the queue
into a string.
    #
    #  It shows the queue's name and type as well as the maximum
number of
    #  items and queue size.
    def __repr__ (self):
        return ('{:<12s} Queue<{:s}> Max Full {:d}/{:d}'.format
(self._name,
                type_code_strings[self._type_code],
self._max_full, self._size))



#
=================================================================
============


## An item which holds data to be shared between tasks.
```

```
#   This class implements a shared data item which can be protected against

#  data corruption by pre-emptive multithreading. Multithreading which can

#  corrupt shared data includes the use of ordinary interrupts as well as the

#   use of pre-emptive multithreading such as by a Real-Time Operating System

#  (RTOS).

#

#  An example of the creation and use of a share is as follows:

#  @code

#  import task_share

#

#  # This share holds a signed short (16-bit) integer

#  my_share = task_share.Queue ('h', name="My Share")

#

#  # Somewhere in one task, put data into the share

#  my_share.put (some_data)

#

#  # In another task, read data from the share

#  something = my_share.get ()

#  @endcode

class Share (BaseShare):


    ## A counter used to give serial numbers to shares for diagnostic use.

    ser_num = 0



    ## Create a shared data item used to transfer data between tasks.
```

```
    #
    #   This method allocates memory in which the shared data will be buffered.
    #
    #   Each share can only carry data of one particular type which must be
    #    chosen from the following list. The data type is specified by a
    #  one-letter type code which is given as for the Python @c array.array
    #  type, which can be any of the following:
    #  |      |      |      |
    #  |:-----|:-----|:-----|
    #   | **b** (signed char) | **B** (unsigned char) | 8 bit integers |
    #  | **h** (signed short) | **H** (unsigned short) | 16 bit integers |
    #   | **i** (signed int) | **I** (unsigned int) | 32 bit integers (probably) |
    #   | **l** (signed long) | **L** (unsigned long) | 32 bit integers |
    #  | **q** (signed long long) | **Q** (unsigned long long) | 64 bit integers |
    #  | **f** (float) | **d** (double-precision float) | |
    #
    #  @param type_code The type of data items which the share can hold
    #  @param thread_protect True if mutual exclusion protection is used
    #  @param name A short name for the share, default @c ShareN where @c N
    #         is a serial number for the share
    def __init__ (self, type_code, thread_protect = True, name = None):
        # First call the parent class initializer
```

```
        super ().__init__ (type_code, thread_protect, name)

        self._buffer = array.array (type_code, [0])

        self._name = str (name) if name != None \
            else 'Share' + str (Share.ser_num)
        Share.ser_num += 1



    ## Write an item of data into the share.
    #
    #  This method puts data into the share; any old data is overwritten.
    #  This code disables interrupts during the writing so as to prevent
    #  data corrupting by an interrupt service routine which might access
    #  the same data.
    #  @param data The data to be put into this share
    #  @param in_ISR Set this to True if calling from within an ISR
    @micropython.native
    def put (self, data, in_ISR = False):

        # Disable interrupts before writing the data
        if self._thread_protect and not in_ISR:
            irq_state = pyb.disable_irq ()

        self._buffer[0] = data

        # Re-enable interrupts
```

```
        if self._thread_protect and not in_ISR:
            pyb.enable_irq (irq_state)



    ## Read an item of data from the share.
    #
    #  If thread protection is enabled, interrupts are disabled during the time
    #   that the data is being read so as to prevent data corruption by changes
    #  in the data as it is being read.
    #  @param in_ISR Set this to True if calling from within an ISR
    @micropython.native
    def get (self, in_ISR = False):
        # Disable interrupts before reading the data
        if self._thread_protect and not in_ISR:
            irq_state = pyb.disable_irq ()

        to_return = self._buffer[0]

        # Re-enable interrupts
        if self._thread_protect and not in_ISR:
            pyb.enable_irq (irq_state)

        return (to_return)



    ## Puts diagnostic information about the share into a string.
    #
```

```
    #   Shares are pretty simple, so we just put the name and
type.
    def __repr__ (self):
        return ("{:<12s} Share<{:s}>".format (self._name,
                type_code_strings[self._type_code]))
```

cotask.py:

```
### @file cotask.py

#   This file contains classes to run cooperatively scheduled tasks in a

#  multitasking system.

#

#    Tasks are created as generators, functions which have infinite loops and call

#   @c yield at least once in the loop. References to all the tasks to be run

#   in the system are kept in a list maintained by class @c CoTaskList; the

#    system scheduler then runs the tasks' @c run() methods according to a

#   chosen scheduling algorithm such as round-robin or highest-priority-first.

#

#  @author JR Ridgely

#  @date   2017-Jan-01 JRR Approximate date of creation of file

#   @date    2021-Dec-18 JRR Docstrings modified to work without DoxyPyPy

#   @copyright This program is copyright (c) 2017-2023 by JR Ridgely and

#                released under the GNU Public License, version 3.0.

#

#  It is intended for educational use only, but its use is not limited thereto.

#    THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"

#   AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE

#    IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
```

ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE

LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR

CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF

SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS

INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN

CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)

ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE

POSSIBILITY OF SUCH DAMAGE.

```
import gc                                   # Memory allocation garbage collector

import utime                                # Micropython version of time library

import micropython                          # This shuts up incorrect warnings
```

Implements multitasking with scheduling and some performance logging.

#

This class implements behavior common to tasks in a cooperative

multitasking system which runs in MicroPython. The ability to be scheduled

on the basis of time or an external software trigger or interrupt is

implemented, state transitions can be recorded, and run times can be

```
#   profiled. The user's task code must be implemented in a generator which
#   yields the state (and the CPU) after it has run for a short and bounded
#  period of time.
#
#  @b Example:
#    @code
#       def task1_fun ():
#              '''! This function switches states repeatedly for no reason '''
#            state = 0
#            while True:
#                if state == 0:
#                    state = 1
#                elif state == 1:
#                    state = 0
#                yield (state)
#
#         # In main routine, create this task and set it to run twice per second
#            task1 = cotask.Task (task1_fun, name = 'Task 1', priority = 1,
#                                period = 500, profile = True, trace = True)
#
#         # Add the task to the list (so it will be run) and run scheduler
#        cotask.task_list.append (task1)
#        while True:
#            cotask.task_list.pri_sched ()
#    @endcode
```

```
class Task:

    ## Initialize a task object so it may be run by the scheduler.
    #
    #  This method initializes a task object, saving copies of constructor
    #  parameters and preparing an empty dictionary for states.
    #
    #  @param run_fun The function which implements the task's code. It must
    #         be a generator which yields the current state.
    #  @param name The name of the task, by default @c NoName. This should
    #         be overridden with a more descriptive name by the programmer.
    #  @param priority The priority of the task, a positive integer with
    #         higher numbers meaning higher priority (default 0)
    #  @param period The time in milliseconds between runs of the task if it's
    #         run by a timer or @c None if the task is not run by a timer.
    #         The time can be given in a @c float or @c int; it will be
    #         converted to microseconds for internal use by the scheduler.
    #  @param profile Set to @c True to enable run-time profiling
    #  @param trace Set to @c True to generate a list of transitions between
    #             states. @b Note: This slows things down and allocates memory.
    #  @param shares A list or tuple of shares and queues used by this task.
```

```
    #           If no list is given, no shares are passed to the task

    def __init__(self, run_fun, name="NoName", priority=0, period=None,

                 profile=False, trace=False, shares=()):

        # The function which is run to implement this task's code. Since it

        # is a generator, we "run" it here, which doesn't actually run it but

        # gets it going as a generator which is ready to yield values

        if shares:

            self._run_gen = run_fun(shares)

        else:

            self._run_gen = run_fun()


        ## The name of the task, hopefully a short and descriptive string.

        self.name = name


        ## The task's priority, an integer with higher numbers meaning higher

        # priority.

        self.priority = int(priority)


        ## The period, in milliseconds, between runs of the task's @c run()

        # method. If the period is @c None, the @c run() method won't be run

        # on a time basis but will instead be run by the scheduler as soon

        # as feasible after code such as an interrupt handler calls the

        # @c go() method.
```

```
        if period != None:
            self.period = int(period * 1000)
            self._next_run = utime.ticks_us() + self.period
        else:
            self.period = period
            self._next_run = None

        # Flag which causes the task to be profiled, in which the execution
        #  time of the @c run() method is measured and basic statistics kept.
        self._prof = profile
        self.reset_profile()

        # The previous state in which the task last ran. It is used to watch
        # for and track state transitions.
        self._prev_state = 0

        # If transition tracing has been enabled, create an empty list in
        # which to store transition (time, to-state) stamps
        self._trace = trace
        self._tr_data = []
        self._prev_time = utime.ticks_us()

        ## Flag which is set true when the task is ready to be run by the
        #  scheduler
        self.go_flag = False
```

```
    ## This method is called by the scheduler; it attempts to
run this task.
    #  If the task is not yet ready to run, this method returns
@c False
    #  immediately; if this task is ready to run, it runs the
task's generator
    #  up to the next @c yield() and then returns @c True.
    #
    #  @return @c True if the task ran or @c False if it did not
    def schedule(self) -> bool:
        if self.ready():

            # Reset the go flag for the next run
            self.go_flag = False

            # If profiling, save the start time
            if self._prof:
                stime = utime.ticks_us()

            # Run the method belonging to the state which should
be run next
            curr_state = next(self._run_gen)

            # If profiling or tracing, save timing data
            if self._prof or self._trace:
                etime = utime.ticks_us()

            # If profiling, save timing data
            if self._prof:
                self._runs += 1
                runt = utime.ticks_diff(etime, stime)
```

```
                if self._runs > 2:
                    self._run_sum += runt
                    if runt > self._slowest:
                        self._slowest = runt

            #  If  transition  logic  tracing  is  on,  record  a
transition; if not,
            # ignore the state. If out of memory, switch tracing
off and
            # run the memory allocation garbage collector
            if self._trace:
                try:
                    if curr_state != self._prev_state:
                        self._tr_data.append(
                            (utime.ticks_diff(etime,
self._prev_time),
                             curr_state))
                except MemoryError:
                    self._trace = False
                    gc.collect()

                self._prev_state = curr_state
                self._prev_time = etime

            return True

        else:
            return False


    ## This method checks if the task is ready to run.
```

```
    #  If the task runs on a timer, this method checks what time
it is; if not,

    #  this method checks the flag which indicates that the task
is ready to

    #  go. This method may be overridden in descendent classes
to implement

    #  some other behavior.

    @micropython.native

    def ready(self) -> bool:

        # If this task uses a timer, check if it's time to run
run() again. If

        # so, set go flag and set the timer to go off at the
next run time

        if self.period != None:

            late       =       utime.ticks_diff(utime.ticks_us(),
self._next_run)

            if late > 0:

                self.go_flag = True

                self._next_run = utime.ticks_diff(self.period,

                                                  -
self._next_run)


                # If keeping a latency profile, record the data

                if self._prof:

                    self._late_sum += late

                    if late > self._latest:

                        self._latest = late


        # If the task doesn't use a timer, we rely on go_flag to
signal ready

        return self.go_flag
```

```
    ## This method sets the period between runs of the task to the given
    #   number of milliseconds, or @c None if the task is triggered by calls
    #  to @c go() rather than time.
    #  @param new_period The new period in milliseconds between task runs
    def set_period(self, new_period):
        if new_period is None:
            self.period = None
        else:
            self.period = int(new_period) * 1000



    ## This method resets the variables used for execution time profiling.
    #  This method is also used by @c __init__() to create the variables.
    def reset_profile(self):
        self._runs = 0
        self._run_sum = 0
        self._slowest = 0
        self._late_sum = 0
        self._latest = 0



    ##  This method returns a string containing the task's transition trace.
    #  The trace is a set of tuples, each of which contains a time and the
    #  states from and to which the system transitioned.
    #   @return A possibly quite large string showing state transitions
```

```
    def get_trace(self):
        tr_str = 'Task ' + self.name + ':'
        if self._trace:
            tr_str += '\n'
            last_state = 0
            total_time = 0.0
            for item in self._tr_data:
                total_time += item[0] / 1000000.0
                tr_str += '{: 12.6f}: {: 2d} -> {:d}\n'.format
(total_time,
                    last_state, item[1])
                last_state = item[1]
        else:
            tr_str += ' not traced'
        return tr_str



    ## Method to set a flag so that this task indicates that
it's ready to run.
    #   This method may be called from an interrupt service
routine or from
    #   another task which has data that this task needs to
process soon.
    def go(self):
        self.go_flag = True



    ## This method converts the task to a string for diagnostic
use.
    #  It shows information about the task, including execution
time
    #  profiling results if profiling has been done.
```

```
    #  @returns The string which represents the task
    def __repr__(self):
        rst = f"{self.name:<16s}{self.priority: 4d}"
        try:
            rst += f"{(self.period / 1000.0): 10.1f}"
        except TypeError:
            rst += '         -'
        rst += f"{self._runs: 8d}"

        if self._prof and self._runs > 0:
            avg_dur = (self._run_sum / self._runs) / 1000.0
            avg_late = (self._late_sum / self._runs) / 1000.0
            rst += f"{avg_dur: 10.3f}{(self._slowest / 1000.0): 10.3f}"
            if self.period != None:
                rst += f"{avg_late: 10.3f}{(self._latest / 1000.0): 10.3f}"
        return rst



# ==================================================================================


## A list of tasks used internally by the task scheduler.
#  This class holds the list of tasks which will be run by the task scheduler.
#  The task list is usually not directly used by the programmer except when
#  tasks are added to it and the scheduler is called. An example showing the
#  use of the task list is given in the last few lines of the documentation
```

```
#  for class @c Task.
#
#  The task list is sorted by priority so that the scheduler can efficiently
#  look through the list to find the highest priority task which is ready to
#   run at any given time. Tasks can also be scheduled in a simpler
#  "round-robin" fashion.
class TaskList:


    ##  Initialize the task list. This creates the list of priorities in
    #  which tasks will be organized by priority.
    def __init__(self):


        ## The list of priority lists. Each priority for which at least one
        #  task has been created has a list whose first element is a task
        #  priority and whose other elements are references to task objects at
        #  that priority.
        self.pri_list = []



    ## Append a task to the task list. The list will be sorted by task
    #  priorities so that the scheduler can quickly find the highest priority
    #  task which is ready to run at any given time.
    #  @param task The task to be appended to the list
    def append(self, task):
```

```
        # See if there's a tasklist with the given priority in
the main list
        new_pri = task.priority
        for pri in self.pri_list:
            # If a tasklist with this priority exists, add this
task to it.
            if pri[0] == new_pri:
                pri.append(task)
                break

        # If the priority isn't in the list, this else clause
starts a new
        # priority list with this task as first one. A priority
list has the
        # priority as element 0, an index into the list of tasks
(used for
        # round-robin scheduling those tasks) as the second
item, and tasks
        # after those
        else:
            self.pri_list.append([new_pri, 2, task])

        # Make sure the main list (of lists at each priority) is
sorted
        self.pri_list.sort(key=lambda pri: pri[0], reverse=True)


    ## Run tasks in order, ignoring the tasks' priorities.
    #
    #   This scheduling method runs tasks in a round-robin
fashion. Each
    #  time it is called, it goes through the list of tasks and
gives each of
```

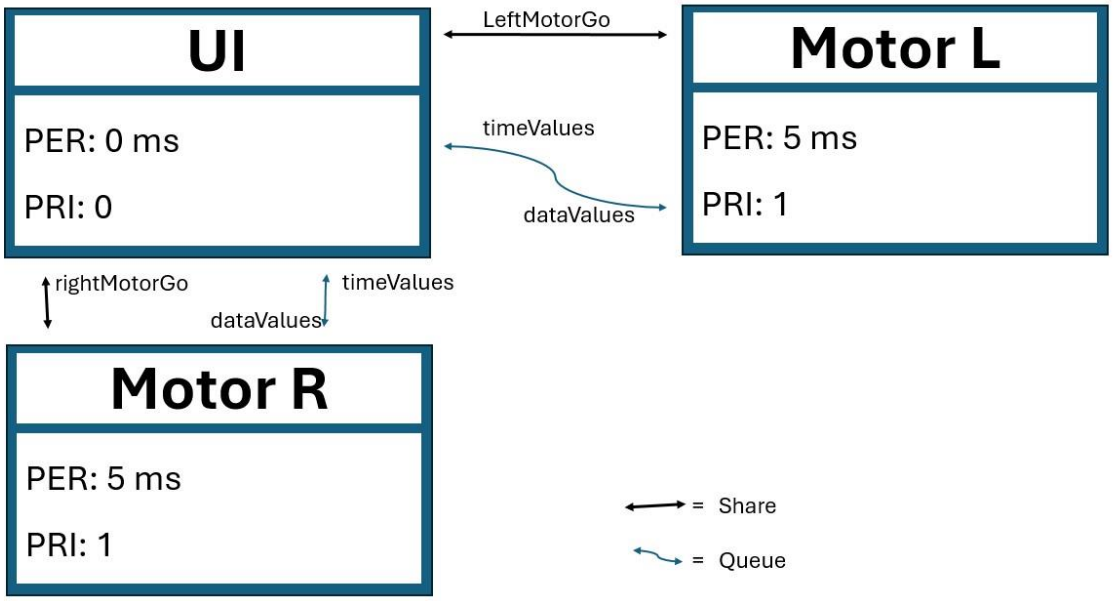
```
    #  them a chance to run. Although this scheduler first runs higher priority
    #  tasks first, that has no significant effect in the long run, as all the
    #  tasks are given a chance to run each time through the list, and it takes
    #  about the same amount of time before each is given a chance to run
    #  again.
    @micropython.native
    def rr_sched(self):
        # For each priority level, run all tasks at that level
        for pri in self.pri_list:
            for task in pri[2:]:
                task.schedule()



    ## Run tasks according to their priorities.
    #
    #  This scheduler runs tasks in a priority based fashion. Each time it is
    #  called, it finds the highest priority task which is ready to run and
    #  calls that task's @c run() method.
    @micropython.native
    def pri_sched(self):
        # Go down the list of priorities, beginning with the highest
        for pri in self.pri_list:
            # Within each priority list, run tasks in round-robin order
            # Each priority list is [priority, index, task, task, ...] where
```

```
            # index is the index of the next task in the list to
be run
            tries = 2
            length = len(pri)
            while tries < length:
                ran = pri[pri[1]].schedule()
                tries += 1
                pri[1] += 1
                if pri[1] >= length:
                    pri[1] = 2
                if ran:
                    return



    ## Create some diagnostic text showing the tasks in the task
list.
    def __repr__(self):
        ret_str = 'TASK              PRI    PERIOD    RUNS   AVG
DUR   MAX ' \
            'DUR  AVG LATE  MAX LATE\n'
        for pri in self.pri_list:
            for task in pri[2:]:
                ret_str += str(task) + '\n'

        return ret_str



## This is @b the main task list which is created for scheduling
when
#  @c cotask.py is imported into a program.
task_list = TaskList()
```

motor_driver.py:

```
## -*- coding: utf-8 -*-
"""
Created on Tue Jan 27 09:37:01 2026

@author: orlen
"""

from pyb import Pin, Timer

class motor_driver:
    def __init__(self, PWM, DIR, nSLP):
        self.pwm=PWM
        self.nSLP_pin = Pin(nSLP, mode=Pin.OUT_PP, value=0)
        self.DIR_pin=Pin(DIR, mode=Pin.OUT_PP, value=0)
        self.nSLP_pin.low()
        self.pwm.pulse_width_percent(0)

    def set_effort(self, effort):
        if effort < -100:
            effort = -100
        elif effort > 100:
            effort = 100

        if effort < 0:
            self.DIR_pin.high()
            effort=-effort
        elif effort >= 0:
            self.DIR_pin.low()
```

```
        self.pwm.pulse_width_percent(int(effort))

    def enable(self):
        self.set_effort(0)
        self.nSLP_pin.high()

    def disable(self):
        self.nSLP_pin.low()
        self.set_effort(0)
```

Attachment B:

Task Diagram: